

5.4-5.10 第 2 周周报

主题：LSTM/GRU 门控机制学习、TinyTensor 项目开发测试与任务一报告撰写

一、任务列表与本周完成情况

本周围绕阶段一的收尾工作和阶段二的前期准备展开。阶段一方面，重点完成 LSTM、GRU 序列模型知识补充、TinyTensor 项目开发测试、任务一报告撰写和 GitHub 开源同步。阶段二方面，开始明确下周文本分类任务的学习与实验方向。

阶段	任务内容	时间节点	本周完成情况	当前状态
阶段一	学习 LSTM 和 GRU 的基本结构与门控机制	5.4-5.10	完成学习笔记，梳理 RNN 长期依赖问题、LSTM 三个门、GRU 两个门及二者对比。	已完成
阶段一	TinyTensor 项目开发与核心模块实现	5.4-5.10	完成自动求导、Module、核心层、损失函数、优化器、DataLoader、Checkpoint 等模块。	已完成
阶段一	TinyTensor 项目测试与应用验证	5.4-5.10	完成 Iris MLP 和 MNIST AlexNet 风格 CNN 实验，记录 loss、accuracy 等结果。	已完成
阶段一	任务一任务报告撰写	5.4-5.10	完成任务一报告，整理框架设计、接口关系、模块实现、实验结果和经验教训。	已完成
阶段一	项目代码开源与仓库同步	5.4-5.10	TinyTensor 代码已开源至 GitHub，实习总仓库同步相关代码、报告和笔记。	已完成
阶段一	根据老师反馈补充任务一实验	5.11 以后	需补充优化器与超参数对比实验，并更新代码、报告和 GitHub 仓库。	待完成
阶段二	任务二文本分类任务准备	5.11 以后	下周学习 NLP 基础，完成文本分类多架构实验、性能比较和任务报告。	待完成

二、本周工作内容概述

- 学习了 LSTM 和 GRU 的基本结构，重点理解二者通过门控机制选择性保留、遗忘和更新历史信息，从而缓解普通 RNN 中长期依赖建模困难的问题。
- 完成了 TinyTensor 项目的开发、测试和整理工作，使其形成了较完整的轻量级神经网络运算框架。
- 使用 Iris 鸢尾花分类和 MNIST 手写数字识别两个应用项目验证 TinyTensor 的基础训练闭环和卷积网络训练能力。
- 完成任务一报告撰写，围绕框架设计、核心模块实现、实验结果、测试验证和经验教训进行了总结。
- 将项目代码开源至 GitHub 仓库，并在实习总仓库中同步本周相关代码、笔记和报告材料。
- 根据老师反馈，明确下周需要进一步补充不同优化器和超参数对训练收敛速度、最终效果的影响分析。

三、本周工作内容详细情况

1. LSTM 和 GRU 门控机制学习

本周首先补充学习了 LSTM 和 GRU 的基本结构。普通 RNN 在长序列建模中存在长期依赖困难，主要原因是反向传播时梯度需要沿时间步进行多次连乘。当连乘项长期小于 1 时容易出现梯度消失，当连乘项长期大于 1 时容易出现梯度爆炸，因此早期时间步的信息很难稳定影响后续较远时间步。

LSTM 在普通 RNN 的隐藏状态之外，引入了细胞状态 c_t 作为长期信息传递通道，并通过遗忘门、输入门和输出门分别控制旧信息保留、新信息写入和当前信息输出。其中，细胞状态更新可以概括为： $c_t = f_t \otimes c_{t-1} + i_t \otimes \tilde{c}_t$ 。该更新方式使长期信息不再完全依赖反复非线性压缩，而是可以沿着相对稳定的加法路径传播。

GRU 可以看作 LSTM 的简化结构。它没有单独的细胞状态，而是使用隐藏状态 h_t 同时承担记忆和输出功能。GRU 主要包含更新门和重置门：更新门控制过去信息与当前候选信息的融合比例，重置门控制生成候选隐藏状态时使用多少历史信息。相比 LSTM，GRU 结构更简洁、参数更少，适合在计算资源有限或希望快速训练的场景中使用。

2. TinyTensor 项目开发测试

本周正式完成 TinyTensor 项目的主体开发。项目定位为一个基于 NumPy 的轻量级神经网络运算框架，目标是在不调用 PyTorch、TensorFlow 等现成深度学习框架的前提下，实现神经网络训练所需的核心能力。项目核心模块包括 TTensor 自动求导、nn.Module 参数管理、常见网络层、损失函数、优化器、数据加载器和 Checkpoint 保存恢复机制。

其中，TTensor 是自动求导的核心数据结构，负责保存 data、grad、计算图前驱节点和局部反向传播函数。调用 `loss.backward()` 后，框架会根据计算图拓扑顺序反向传播梯度，并将梯度累积到可训练参数上。Module 基类则负责递归收集参数和 buffer，使 Linear、Conv2d、BatchNorm、Pooling、Dropout 等层能够以统一方式组合成模型。

卷积层是本周实现中的重点模块。最初的朴素卷积实现虽然直观，但 Python 多重循环在 MNIST 卷积网络上效率较低。最终代码将 Conv2d 优化为 `im2col/col2im + NumPy 矩阵乘法` 的形式，把主要计算交给 NumPy 的矩阵运算后端完成，从而在保持接口不变的情况下显著减少 Python 层循环开销。

3. 应用项目验证与任务一报告撰写

为了验证 TinyTensor 不是只能完成单个算子测试，本周使用两个应用项目对框架进行了验证。第一个项目是 Iris 鸢尾花分类，用于验证 Linear、LayerNorm、ReLU、CrossEntropyLoss、DataLoader、Optimizer 和 Checkpoint 组成的基础训练闭环；第二个项目是 MNIST 手写数字识别，用于验证 Conv2d、Pooling、LRN、AdaptiveAvgPool、Dropout、Flatten、Linear 等模块在较完整 CNN 中的协同工作。

在任务一报告中，我对项目的完成情况、总体设计、目录结构、关键接口关系、核心模块实现、卷积层与池化层实现、应用验证结果、测试验证与可复现性进行了系统整理。报告中的软件架构图和接口关系表也帮助我进一步梳理了 TinyTensor 中各类之间的调用关系，例如训练脚本通过 `model(x)` 得到 logits，使用 `loss` 计算训练目标，再通过 `loss.backward()` 和 `optimizer.step()` 完成参数更新。

4. 老师反馈与问题整理

老师对任务一阶段成果给出的反馈主要包括两方面：一方面，代码较规范，工程层面相对完善，项目已经 GitHub 开源，报告中代码的类关系和接口说明也比较清晰；另一方面，代码中实现了 SGD、Adam、AdamW 等多种优化器方案，但报告中实验部分只重点介绍了一种实验结果，还可以补充不同优化器和不同超参数对模型收敛速度与最终效果的影响。

因此，下周除完成任务二外，还需要对任务一进行补充完善。具体来说，需要固定数据集、模型结构和随机种子，在尽量控制变量的前提下比较 SGD、Adam、AdamW 等优化器的训练损失下降速度、验证准确率、验证损失和训练耗时，并对学习率、batch size、weight decay 等关键超参数进行适当对比。完成后需要同步修改任务一报告、项目代码和 GitHub 仓库。

四、本周进展与计划符合情况

本周整体进展基本符合上周计划。上周计划中要求学习 LSTM 和 GRU、开展 TinyTensor 项目开发、对核心模块进行测试、使用简单数据集验证训练流程、整理 README 和实验结果、撰写任务一报告并提交阶段性成果。本周已经完成 LSTM/GRU 笔记整理、TinyTensor 主体代码开发、Iris 与 MNIST 两个应用项目验证、任务一报告撰写和 GitHub 开源同步。

需要说明的是，任务一虽然已经完成主要开发与报告撰写，但根据老师反馈，实验分析部分仍有进一步完善空间。后续需要把“能跑通模型”推进到“能比较和讨论不同训练策略”，使任务报告更能体现实验设计、对比分析和工程验证能力。

五、本周收获与思考

本周最大的收获之一是进一步理解了序列模型中门控机制的意义。LSTM 和 GRU 的核心并不是简单增加参数，而是通过可学习的门控变量控制信息的保留、遗忘和更新，使模型能够在较长时间跨度内选择性保存有用历史信息。这一点也帮助我更好地理解后续 NLP 文本分类任务中为什么会使用 RNN、LSTM、GRU 或注意力机制处理上下文信息。

第二个收获是对深度学习框架的工程组织有了更具体的认识。一个可用的训练框架不仅需要实现网络层，还需要同时考虑自动求导、参数管理、训练/推理模式切换、优化器状态、数据加载、checkpoint 保存恢复和实验结果记录。如果这些接口没有统一设计，后续扩展新模型和复现实验都会变得困难。

第三个收获是认识到任务报告不能只停留在“实现了哪些模块”。老师反馈提醒我，实验分析应当更充分地回答“不同设计选择会带来什么影响”。例如代码中已经实现了多种优化器，那么报告中就应该比较它们在收敛速度、训练稳定性和最终性能上的差异，而不是只展示单一实验曲线。

六、下周工作计划

1. 任务二文本分类与 NLP 基础学习

在任务二的文本分类任务驱动下，学习 NLP 领域的基本概念，包括文本预处理、分词、词表构建、词向量表示、序列建模、文本分类评价指标等。

梳理文本分类常见模型架构，重点关注传统特征方法、MLP、TextCNN、RNN/LSTM/GRU 以及基于注意力或 Transformer 思想的模型。

使用几种不同架构完成文本分类实验，并从准确率、损失变化、收敛速度、模型复杂度等角度进行性能比较和讨论。

整理任务二实验数据、训练曲线和结果表格，撰写任务二任务报告。将任务二相关代码、学习笔记和实验数据同步开源至 GitHub 仓库，并将项目代码和任务报告提交给老师。

2. 根据老师反馈补充任务一实验与报告

待改进点	具体修改计划
优化器对比不足	补充 SGD、Adam、AdamW 等优化器在同一数据集和同一模型结构下的对比实验，观察训练损失下降速度、验证准确率、验证损失和训练耗时。
超参数对比不足	在控制变量的前提下比较学习率、batch size、weight decay 等超参数对收敛速度和最终效果的影响。
报告实验分析不够充分	在任务一报告中新增优化器与超参数对比小节，加入结果表格、曲线图和现象解释，避免只展示单一实验结果。
GitHub 材料需要同步更新	将补充实验代码、实验数据、更新后的任务一报告和 README 同步提交至 GitHub 仓库，并将修改后的项目代码和任务报告提交给老师。

七、附录

附录 1：GitHub 仓库链接

线上实习总仓库：https://github.com/stream2005/research_internship

TinyTensor 项目仓库：<https://github.com/stream2005/TinyTensor>